SUNHAVEN CARE IAM

Complete Development Brief for Building the Project

Purpose of this document

Give this PDF to the AI/developer that will build the project. It explains, in simple language, what the system is, what must be built, which technologies to use, how the system should work, what to test, and what must be shown in the final assessment demonstration.

1. What are we building?

A small Employee Identity and Access Management (IAM) system for Sunhaven Care. It is not a payroll or full HR system. Its job is to control who can log in, what each worker can access, how access changes when a worker changes role, and how access is removed when a worker leaves.

Simple idea:

```
Employee joins -> account and correct access are created -> employee changes role -> access is updated -> employee leaves -> access is removed and the user is blocked.
```

Main security problem

Sunhaven has permanent, casual and agency workers, shared devices, sensitive resident information, role changes and frequent leavers. The system must prevent old accounts, excessive permissions, shared-session misuse, and access that continues after a worker leaves.

2. Main project outcomes

- **Joiner:** safely create/reconcile a new worker and give only approved access.
- **Mover:** remove obsolete access before giving the new approved role/access.
- **Leaver:** disable identity, remove access, remove assignments, block portal use, and prove access is denied.
- **RBAC:** enforce least privilege based on role and assignments.
- **MFA:** use Microsoft Entra ID for stronger authentication.
- **Audit evidence:** record who did what, when, to which identity, and whether the action succeeded.
- **Access review:** allow a manager/auditor to retain, change, remove, or temporarily accept access.
- **Compliance checking:** detect obvious policy violations and show PASS/FAIL findings.

3. Requirements the finished project must meet

The following are the practical requirements to build into the final prototype.

Area	Minimum requirement
Identity	Use named individual users. No shared workforce accounts.
Roles	CareWorker, Nurse, Manager, AgencyWorker, Auditor, IAMOperator.
Authentication	Microsoft Entra ID login; MFA for user personas when the tenant/licence allows it.
Authorisation	Server-side role checks. Authentication alone must not grant protected access.
Resident access	Show only fictional/synthetic residents and only when the user has the correct role/assignment.
Joiner	Validate worker data, create/reconcile identity, assign correct role/group, create assignments, verify final state.
Mover	Show intended change, remove old access first, add new role/access, update assignments, verify old access is gone.
Leaver	Disable account, revoke available sessions, remove roles/groups/assignments, block portal, verify protected access is denie
Agency worker	Must have sponsor and end date. Expired agency access must be removed/reviewed.
Access review	Manager/Auditor can Retain, Change, Remove, or Temporary Exception; store reviewer, date, reason and remediation.
Audit	Store target, action, operator, UTC time, result, correlation ID and final-state verification.
Privacy	Never show/store passwords, MFA codes, access tokens, refresh tokens or client secrets in evidence.
Compliance checker	Detect enabled leavers, expired agencies, role mismatch, conflicting roles, disabled users with access, and unexpected adm

Important scope rule

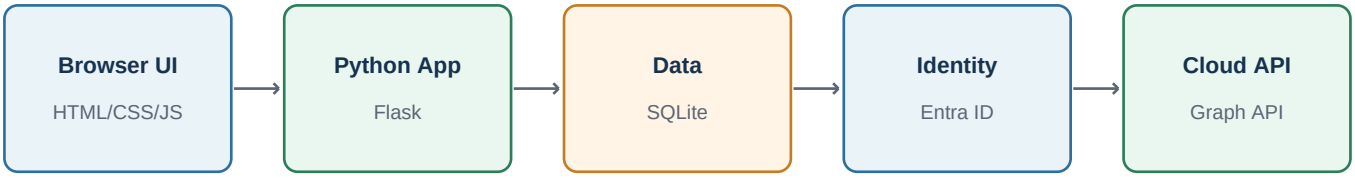
Use only fictional worker and resident data. This is a student laboratory proof of concept, not a production healthcare system.

4. Recommended programming and technology

Use the simplest stack that still meets the assessment requirements. The recommendation below intentionally removes PowerShell.

Layer	Technology	Why
Frontend	HTML + CSS + Bootstrap + small JavaScript	Simple forms, tables, buttons and dashboards.
Backend	Python + Flask	Very small framework; makes routes, forms, sessions and security much easier t
Database	SQLite	Simple local database, no separate database server needed.
Authentication	Microsoft Entra ID	Provides identity, SSO and MFA.
Cloud operations	Microsoft Graph API from Python	Create/update/disable identities and manage access without PowerShell.
Auth library	MSAL for Python	Recommended small library for Microsoft identity authentication/token handling.
Reports/evidence	Python -> JSON/CSV/HTML	Easy to generate repeatable audit/test evidence.
Version control	GitHub	Required evidence of individual/team development and final source history.

Recommended architecture



Recommended final stack: one small web application. No PowerShell.

Design change to document

The revised group proposal originally describes Microsoft Graph PowerShell for JML automation. For this implementation, replace that layer with Python calling Microsoft Graph API. Do not hide the change: explain in the final report that the design evolved to reduce complexity while keeping the same security outcomes.

5. Features and pages to build

A. Login

Sign in with Entra ID. After authentication, read the user's identity and roles. Enforce server-side permissions.

B. Dashboard

Show counts: active workers, leavers/disabled users, agency workers, pending reviews, recent audit events and compliance findings.

C. Employees

List workers. Search/filter. View employee ID, name, worker type, role, facility, status, sponsor/manager, start date and end date.

D. Add Employee / Joiner

Form to create an approved worker. Validate required fields. Save to SQLite. Call Graph API to create/reconcile Entra identity and assign approved access. Record evidence.

E. Edit Employee / Mover

Change role/facility/assignments. Show before/after. Remove obsolete access first, then add new approved access. Verify the old access no longer works.

F. Leaver

Confirm departure. Disable Entra identity, remove governed access, remove local assignments, add user to portal blocked list, and test denial.

G. Residents

Use synthetic residents only. Show records based on role + assignment. Never use real healthcare data.

H. Protected Pages

Example pages such as Care Notes, Clinical, Review and IAM Admin. Each route must check role on the server.

I. Access Review

Manager/Auditor reviews each user and chooses Retain / Change / Remove / Temporary Exception. Store decision, reviewer, reason, date, expiry and remediation state.

J. Audit Log

Sanitised audit table. Search/filter. Include event ID, target, action, operator, UTC time, result, correlation ID and final-state check.

K. Compliance Checker

Run deterministic rules and display PASS/FAIL findings with reason and recommended remediation.

L. 403 / Access Denied

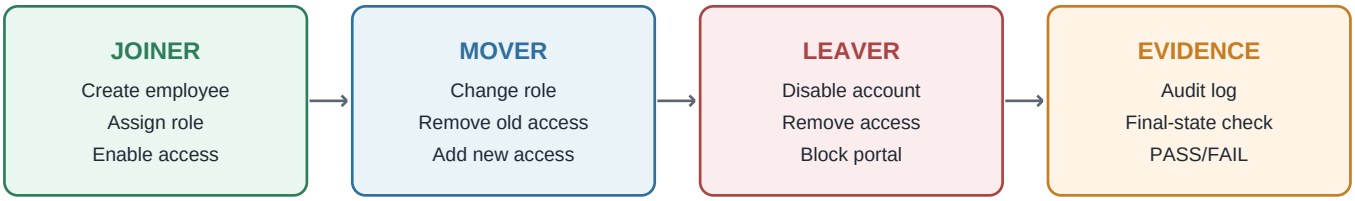
Show a clear denial page and record the denied request in the sanitised audit log.

6. Suggested database design

Keep the database small. The following tables are enough for the prototype.

Table	Main fields	Purpose
employees	employee_id, name, email/upn, worker_type, role, facility, status, manager, start_date, end_date, entra_object_id	Approved worker records.
residents	resident_id, fictional_name, facility	Synthetic resident records.
assignments	employee_id, resident_id, active	Controls which residents a worker may see.
access_reviews	review_id, employee_id, reviewer, decision, reason, review_date, exception_expiry, evidence_status	Manager/auditor review evidence.
blocked_users	employee_id, blocked_at, reason	Immediate local portal denial for leavers/suspended users.
audit_events	event_id, actor, target, action, utc_time, result, correlation_id, final_state	Security and lifecycle evidence.
compliance_findings	finding_id, employee_id, rule_id, severity, status, details, detected_at	Policy-compliance results.

7. How the employee lifecycle finally works



Joiner example: Sarah is approved as a Nurse -> admin enters her record -> Python validates it -> SQLite stores it -> Graph API creates/reconciles her Entra account -> Nurse role is assigned -> permitted pages work -> audit evidence is recorded.

Mover example: Sarah becomes a Manager -> system shows the proposed change -> Nurse access is removed -> Manager access is added -> assignments are updated -> old Nurse-only access is tested and must fail -> evidence is recorded.

Leaver example: Sarah leaves -> account is disabled -> sessions are revoked where supported -> groups/roles/assignments are removed -> portal blocked list is updated -> new sign-in fails and the next protected request from an old session is denied -> final state is logged.

8. Security rules the developer must enforce

- Never trust a role sent from the browser. Authorisation must be checked in Python on every protected route.
- Use a stable employee ID to identify workers. Do not target an account by name alone.
- Reject incomplete, duplicate or ambiguous worker records instead of guessing.
- For movers, remove obsolete access before adding new access.
- For leavers, use layered denial: cloud account disablement + removal of roles/groups + local blocked-user check + assignment removal.
- Use synthetic test data only.
- Do not put passwords, MFA codes, Graph tokens, refresh tokens or client secrets into screenshots, logs or reports.
- Store secrets outside source code (environment variables or local secure configuration ignored by Git).
- Generate a correlation ID for lifecycle operations so actions and evidence can be traced.
- After every important change, read back the final state and record PASS/FAIL.
- Make compliance checking read-only. A finding must not automatically change Entra; remediation requires an explicit approved action.

Do not overbuild

Do NOT build payroll, salary, attendance, rostering, billing, real clinical records, mobile apps, production SIEM, MDM, disaster recovery, or a full HR platform. These are outside the intended prototype.

9. Compliance rules to implement

Rule	FAIL condition
C01 Enabled leaver	status = Leaver/Inactive but Entra/local access is still enabled
C02 Expired agency	agency end date has passed but access remains active
C03 Agency data incomplete	agency worker has no sponsor or no end date
C04 Role mismatch	actual role does not match approved workforce role
C05 Conflicting roles	worker has more than one normal care role without approved exception
C06 Disabled user with access	disabled identity still has governed role/group/assignment
C07 Unexpected privilege	normal worker has IAM/admin privilege
C08 Review overdue	required access review has not been completed in the expected period

10. Testing and acceptance criteria

The developer should create repeatable tests and capture real screenshots/logs from the running system.

ID	Test	Expected result
T01	Create valid Nurse joiner	Account/data created; Nurse access works; evidence logged
T02	Reject invalid/duplicate worker	No unsafe change; clear error
T03	Agency worker without sponsor/end date	Rejected
T04	Authorised resident access	Allowed
T05	Unauthorised protected route	HTTP 403 / denied and audited
T06	Mover Nurse -> Manager	Old role removed, new role added
T07	Verify old mover access	Old restricted access no longer works
T08	Leaver cloud/local disable	Identity/access removed or disabled
T09	Leaver new sign-in	Denied
T10	Leaver existing portal session	Next protected request denied
T11	Logout/session termination	Session ends; protected route requires authentication
T12	Audit evidence	Correct fields, no secrets
T13	Manager access review	Decision saved with reviewer/date/reason
T14	Compliance checker seeded violation	Expected FAIL finding generated
T15	Remediation and re-scan	Finding clears after approved correction

11. Development order

Phase	Build
1	Project folders, configuration, SQLite schema, synthetic seed data
2	Employee CRUD pages and dashboard
3	Local Joiner/Mover/Leaver logic + audit logging
4	Authentication/session handling and server-side RBAC
5	Residents and employee-resident assignments
6	Access review workflow
7	Compliance checker and findings page
8	Microsoft Entra ID login + MFA
9	Microsoft Graph API lifecycle integration in Python
10	End-to-end tests, failure cases, screenshots and evidence
11	README, setup guide, architecture diagram, demo script
12	Final Assessment 3 report and demo video

12. What the final Assessment 3 must show

Assessment 3 is the group final report and working-prototype demonstration. The final report must show the complete scope, design evolution, implementation/prototype, evaluation, project management, ethical/privacy considerations and professional evidence.

Recommended demo sequence (under 10 minutes):

- 1. Explain the Sunhaven problem in 30-45 seconds.
- 2. Log in and show the dashboard.
- 3. Create a Joiner and show correct role/access.
- 4. Perform a Mover and show old access removed and new access working.
- 5. Perform the key Leaver demonstration: disable/remove/block, then prove access is denied.
- 6. Show role-based resident access and a 403 denial.
- 7. Complete one manager access review.
- 8. Run the compliance checker on a seeded violation and show the finding.
- 9. Show the sanitised audit evidence and final-state verification.
- 10. Briefly show GitHub/test results and explain limitations.

13. Deliverables the developer should produce

- Working source code with clear folder structure.
- requirements.txt and environment/config example.
- SQLite schema + seed script with fictional data.
- README with setup and run instructions.
- Architecture diagram and short explanation.
- Role/permission matrix.
- Joiner/Mover/Leaver implementation.
- Entra/MSAL authentication and Python Graph API integration.
- Access review and compliance checker.
- Automated/repeatable tests and test results.
- Evidence folder with real screenshots/logs, not illustrative mock-ups.
- Demo runbook/script for the final video.
- Final report material: design decisions, limitations, security/privacy, testing, project evolution and evaluation.

Instruction to the Work AI / developer

Build the complete Sunhaven Care IAM prototype described in this document.

Use Python + Flask + SQLite + HTML/CSS/Bootstrap/JavaScript. Use Microsoft Entra ID for authentication/MFA and use Microsoft Graph API directly from Python for identity/access operations. Do not use PowerShell. Keep the implementation simple, modular and demonstrable. Use synthetic data only. Implement Joiner, Mover, Leaver, RBAC, resident assignments, blocked-user checks, audit logging, access review and a deterministic compliance checker. Every security-critical feature must have a repeatable test and real evidence. Do not claim a feature as completed until it runs and has been tested. Preserve the project requirements even if internal implementation details are simplified.

14. Source basis

This brief is based on the team's supplied project and assessment material, including:

- COIT13236 Assessment 3 Specifications, T2 2026.
- Sunhaven Care Workforce IAM Revised Group Project Proposal (26 August 2026).
- Sunhaven Care Security Policy and Procedure Set v2.0 Draft (9 September 2026).
- POL-01 Access Management Policy.
- Assessment 1 and Assessment 2 specifications and the team's existing technical artefacts.

Implementation recommendation added in this brief: replace the proposal's PowerShell JML layer with Python + Microsoft Graph API to reduce complexity. This is a design evolution and must be documented transparently in the final report.